In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
```

In [2]:
```
import os
import joblib
```

In [3]:
```
ml_logs_dir = 'ML logs'
if not os.path.exists(ml_logs_dir):
    os.makedirs(ml_logs_dir)

# Define subdirectories for models and graphs
models_dir = os.path.join(ml_logs_dir, 'models')
graphs_dir = os.path.join(ml_logs_dir, 'graphs')

if not os.path.exists(models_dir):
    os.makedirs(models_dir)

if not os.path.exists(graphs_dir):
    os.makedirs(graphs_dir)
```

In [4]:
```
Data="C:/Users/Tharunraj_S/Desktop/Mini_project\kidney_disease.csv"
df=pd.read_csv(Data)
df.head()
```

Out[4]:

	id	age	bp	sg	al	su	rbc	pc	pcc	ba	...	pcv	wc
0	0	48.0	80.0	1.020	1.0	0.0	NaN	normal	notpresent	notpresent	...	44	7800
1	1	7.0	50.0	1.020	4.0	0.0	NaN	normal	notpresent	notpresent	...	38	6000
2	2	62.0	80.0	1.010	2.0	3.0	normal	normal	notpresent	notpresent	...	31	7500
3	3	48.0	70.0	1.005	4.0	0.0	normal	abnormal	present	notpresent	...	32	6700
4	4	51.0	80.0	1.010	2.0	0.0	normal	normal	notpresent	notpresent	...	35	7300

5 rows × 26 columns

In [5]:
```
df.shape
```

Out[5]:
```
(400, 26)
```

In [6]:
```
df.columns
```

Out[6]:
```
Index(['id', 'age', 'bp', 'sg', 'al', 'su', 'rbc', 'pc', 'pcc', 'ba',
'bgr',
       'bu', 'sc', 'sod', 'pot', 'hemo', 'pcv', 'wc', 'rc', 'htn', 'd
m', 'cad',
       'appet', 'pe', 'ane', 'classification'],
      dtype='object')
```

In [7]:
```
# dropping id column
df.drop('id', axis = 1, inplace = True)
```

In [8]:
```
# rename column names to make it more meaningful
df.columns = ['age', 'blood_pressure', 'specific_gravity', 'albumin', '
              'pus_cell_clumps', 'bacteria', 'blood_glucose_random', 'b
              'potassium', 'haemoglobin', 'packed_cell_volume', 'white_
              'hypertension', 'diabetes_mellitus', 'coronary_artery_dis
              'anemia', 'class']
```

In [9]:
```
df.columns
```

Out[9]:
```
Index(['age', 'blood_pressure', 'specific_gravity', 'albumin', 'suga
r',
       'red_blood_cells', 'pus_cell', 'pus_cell_clumps', 'bacteria',
       'blood_glucose_random', 'blood_urea', 'serum_creatinine', 'sodi
um',
       'potassium', 'haemoglobin', 'packed_cell_volume',
       'white_blood_cell_count', 'red_blood_cell_count', 'hypertensio
n',
       'diabetes_mellitus', 'coronary_artery_disease', 'appetite',
       'peda_edema', 'anemia', 'class'],
      dtype='object')
```

In [10]:
```
df.describe()
```

Out[10]:

	age	blood_pressure	specific_gravity	albumin	sugar	blood_gluc
count	391.000000	388.000000	353.000000	354.000000	351.000000	
mean	51.483376	76.469072	1.017408	1.016949	0.450142	
std	17.169714	13.683637	0.005717	1.352679	1.099191	
min	2.000000	50.000000	1.005000	0.000000	0.000000	
25%	42.000000	70.000000	1.010000	0.000000	0.000000	
50%	55.000000	80.000000	1.020000	0.000000	0.000000	
75%	64.500000	80.000000	1.020000	2.000000	0.000000	
max	90.000000	180.000000	1.025000	5.000000	5.000000	

In [11]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 25 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   age                       391 non-null    float64
 1   blood_pressure            388 non-null    float64
 2   specific_gravity          353 non-null    float64
 3   albumin                   354 non-null    float64
 4   sugar                     351 non-null    float64
 5   red_blood_cells           248 non-null    object 
 6   pus_cell                  335 non-null    object 
 7   pus_cell_clumps           396 non-null    object 
 8   bacteria                  396 non-null    object 
 9   blood_glucose_random      356 non-null    float64
 10  blood_urea                381 non-null    float64
 11  serum_creatinine          383 non-null    float64
 12  sodium                    313 non-null    float64
 13  potassium                 312 non-null    float64
 14  haemoglobin               348 non-null    float64
 15  packed_cell_volume        330 non-null    object 
 16  white_blood_cell_count    295 non-null    object 
 17  red_blood_cell_count      270 non-null    object 
 18  hypertension              398 non-null    object 
 19  diabetes_mellitus         398 non-null    object 
 20  coronary_artery_disease   398 non-null    object 
 21  appetite                  399 non-null    object 
 22  peda_edema                399 non-null    object 
 23  anemia                    399 non-null    object 
 24  class                     400 non-null    object 
dtypes: float64(11), object(14)
memory usage: 78.3+ KB
```

In [12]:

```
# Printing unique values in data
for i in df.columns:
    print('unique values in "{}":\n'.format(i),df[i].unique())
```

```
unique values in "age":
 [48.  7. 62. 51. 60. 68. 24. 52. 53. 50. 63. 40. 47. 61. 21. 42. 75.
69.
 nan 73. 70. 65. 76. 72. 82. 46. 45. 35. 54. 11. 59. 67. 15. 55. 44. 2
6.
 64. 56.  5. 74. 38. 58. 71. 34. 17. 12. 43. 41. 57.  8. 39. 66. 81. 1
4.
 27. 83. 30.  4.  3.  6. 32. 80. 49. 90. 78. 19.  2. 33. 36. 37. 23. 2
5.
 20. 29. 28. 22. 79.]
unique values in "blood_pressure":
 [ 80.  50.  70.  90.  nan 100.  60. 110. 140. 180. 120.]
unique values in "specific_gravity":
 [1.02  1.01  1.005 1.015   nan 1.025]
unique values in "albumin":
 [ 1.  4.  2.  3.  0. nan  5.]
unique values in "sugar":
 [ 0.  3.  4.  1. nan  2.  5.]
unique values in "red_blood_cells":
 [nan 'normal' 'abnormal']
unique values in "pus_cell":
 ['normal' 'abnormal' nan]
unique values in "pus_cell_clumps":
 ['notpresent' 'present' nan]
unique values in "bacteria":
 ['notpresent' 'present' nan]
unique values in "blood_glucose_random":
 [121.  nan 423. 117. 106.  74. 100. 410. 138.  70. 490. 380. 208.  9
8.
 157.  76.  99. 114. 263. 173.  95. 108. 156. 264. 123.  93. 107. 159.
 140. 171. 270.  92. 137. 204.  79. 207. 124. 144.  91. 162. 246. 253.
 141. 182.  86. 150. 146. 425. 112. 250. 360. 163. 129. 133. 102. 158.
 165. 132. 104. 127. 415. 169. 251. 109. 280. 210. 219. 295.  94. 172.
 101. 298. 153.  88. 226. 143. 115.  89. 297. 233. 294. 323. 125.  90.
 308. 118. 224. 128. 122. 214. 213. 268. 256.  84. 105. 288. 139.  78.
 273. 242. 424. 303. 148. 160. 192. 307. 220. 447. 309.  22. 111. 261.
 215. 234. 131. 352.  80. 239. 110. 130. 184. 252. 113. 230. 341. 255.
 103. 238. 248. 120. 241. 269. 201. 203. 463. 176.  82. 119.  97.  96.
  81. 116. 134.  85.  83.  87.  75.]
unique values in "blood_urea":
 [ 36.   18.   53.   56.   26.   25.   54.   31.   60.  107.   55.   7
2.
  86.   90.  162.   46.   87.   27.  148.  180.  163.    nan  50.   7
5.
  45.   28.  155.   33.   39.  153.   29.   65.  103.   70.   80.   2
0.
 202.   77.   89.   24.   17.   32.  114.   66.   38.  164.  142.   9
6.
 391.   15.  111.   73.   19.   92.   35.   16.  139.   48.   85.   9
8.
 186.   37.   47.   52.   82.   51.  106.   22.  217.   88.  118.   5
0.1
  71.   34.   40.   21.  219.   30.  125.  166.   49.  208.  176.   6
8.
 145.  165.  322.   23.  235.  132.   76.   42.   44.   41.  113.
1.5
 146.   58.  133.  137.   67.  115.  223.   98.6 158.   94.   74.  15
0.
  61.   57.   95.  191.   93.  241.   64.   79.  215.  309.   10. ]
unique values in "serum_creatinine":
 [ 1.2   0.8   1.8   3.8   1.4   1.1  24.    1.9   7.2   4.    2.7
```

```
2.1
  4.6   4.1   9.6   2.2   5.2   1.3   1.6   3.9  76.    7.7    nan  2.
4
  7.3   1.5   2.5   2.    3.4   0.7   1.   10.8   6.3   5.9   0.9   3.
  3.25  9.7   6.4   3.2  32.    0.6   6.1   3.3   6.7   8.5   2.8  15.
  2.9   1.7   3.6   5.6   6.5   4.4  10.2  11.5   0.5  12.2   5.3   9.
2
 13.8  16.9   6.    7.1  18.    2.3  13.   48.1  14.2  16.4   2.6   7.
5
  4.3  18.1  11.8   9.3   6.8  13.5  12.8  11.9  12.   13.4  15.2  13.
3
  0.4 ]
unique values in "sodium":
 [  nan 111.  142.  104.  114.  131.  138.  135.  130.  141.  139.
4.5
 136.  129.  140.  132.  133.  134.  125.  163.  137.  128.  143.  12
7.
 146.  126.  122.  147.  124.  115.  145.  113.  120.  150.  144. ]
unique values in "potassium":
 [ nan  2.5  3.2  4.   3.7  4.2  5.8  3.4  6.4  4.9  4.1  4.3  5.2  3.
8
  4.6  3.9  4.7  5.9  4.8  4.4  6.6 39.   5.5  5.   3.5  3.6  7.6  2.9
  4.5  5.7  5.4  5.3 47.   6.3  5.1  5.6  3.   2.8  2.7  6.5  3.3]
unique values in "haemoglobin":
 [15.4 11.3  9.6 11.2 11.6 12.2 12.4 10.8  9.5  9.4  9.7  9.8  5.6  7.
6
 12.6 12.1 12.7 10.3  7.7 10.9  nan 11.1  9.9 12.5 12.9 10.1 12.  13.
  7.9  9.3 15.  10.   8.6 13.6 10.2 10.5  6.6 11.   7.5 15.6 15.2  4.8
  9.1  8.1 11.9 13.5  8.3  7.1 16.1 10.4  9.2  6.2 13.9 14.1  6.  11.8
 11.7 11.4 14.   8.2 13.2  6.1  8.  12.3  8.4 14.3  9.   8.7 10.6 13.1
 10.7  5.5  5.8  6.8  8.8  8.5 13.8 11.5  7.3 13.7 12.8 13.4  6.3  3.1
 17.  15.9 14.5 15.5 16.2 14.4 14.2 16.3 14.8 16.5 15.7 13.3 14.6 16.4
 16.9 16.  14.7 16.6 14.9 16.7 16.8 15.8 15.1 17.1 17.2 15.3 17.3 17.4
 17.7 17.8 17.5 17.6]
unique values in "packed_cell_volume":
 ['44' '38' '31' '32' '35' '39' '36' '33' '29' '28' nan '16' '24' '37'
'30'
 '34' '40' '45' '27' '48' '\t?' '52' '14' '22' '18' '42' '17' '46' '2
3'
 '19' '25' '41' '26' '15' '21' '43' '20' '\t43' '47' '9' '49' '50' '5
3'
 '51' '54']
unique values in "white_blood_cell_count":
 ['7800' '6000' '7500' '6700' '7300' nan '6900' '9600' '12100' '4500'
 '12200' '11000' '3800' '11400' '5300' '9200' '6200' '8300' '8400' '10
300'
 '9800' '9100' '7900' '6400' '8600' '18900' '21600' '4300' '8500' '113
00'
 '7200' '7700' '14600' '6300' '\t6200' '7100' '11800' '9400' '5500' '5
800'
 '13200' '12500' '5600' '7000' '11900' '10400' '10700' '12700' '6800'
 '6500' '13600' '10200' '9000' '14900' '8200' '15200' '5000' '16300'
 '12400' '\t8400' '10500' '4200' '4700' '10900' '8100' '9500' '2200'
 '12800' '11200' '19100' '\t?' '12300' '16700' '2600' '26400' '8800'
 '7400' '4900' '8000' '12000' '15700' '4100' '5700' '11500' '5400' '10
800'
 '9900' '5200' '5900' '9300' '9700' '5100' '6600']
unique values in "red_blood_cell_count":
 ['5.2' nan '3.9' '4.6' '4.4' '5' '4.0' '3.7' '3.8' '3.4' '2.6' '2.8'
'4.3'
 '3.2' '3.6' '4' '4.1' '4.9' '2.5' '4.2' '4.5' '3.1' '4.7' '3.5' '6.0'
```

```
 '5.0' '2.1' '5.6' '2.3' '2.9' '2.7' '8.0' '3.3' '3.0' '3' '2.4' '4.8'
 '\t?' '5.4' '6.1' '6.2' '6.3' '5.1' '5.8' '5.5' '5.3' '6.4' '5.7' '5.
9'
 '6.5']
unique values in "hypertension":
 ['yes' 'no' nan]
unique values in "diabetes_mellitus":
 ['yes' 'no' ' yes' '\tno' '\tyes' nan]
unique values in "coronary_artery_disease":
 ['no' 'yes' '\tno' nan]
unique values in "appetite":
 ['good' 'poor' nan]
unique values in "peda_edema":
 ['no' 'yes' nan]
unique values in "anemia":
 ['no' 'yes' nan]
unique values in "class":
 ['ckd' 'ckd\t' 'notckd']
```

In [13]:
```
#PREPROCESSING
```

In [14]:
```
# replace some incorrect values
df['diabetes_mellitus'].replace(to_replace = {'\tno':'no','\tyes':'yes'

df['coronary_artery_disease'] = df['coronary_artery_disease'].replace(t

df['class'] = df['class'].replace(to_replace = {'ckd\t': 'ckd', 'notckd
```

In [15]:
```
# converting necessary columns to numerical type
df['packed_cell_volume'] = pd.to_numeric(df['packed_cell_volume'], erro
df['white_blood_cell_count'] = pd.to_numeric(df['white_blood_cell_count
df['red_blood_cell_count'] = pd.to_numeric(df['red_blood_cell_count'],
```

In [16]:
```
df['class'] = df['class'].map({'not ckd': 0, 'ckd': 1})
df['class'] = pd.to_numeric(df['class'], errors='coerce')
```

In [17]:
```
# Extracting categorical and numerical columns
cat_cols = [col for col in df.columns if df[col].dtype == 'object']
num_cols = [col for col in df.columns if df[col].dtype != 'object']
```

In [18]:
```
df.head(10)
```

Out[18]:

	age	blood_pressure	specific_gravity	albumin	sugar	red_blood_cells	pus_cell	pu
0	48.0	80.0	1.020	1.0	0.0	NaN	normal	
1	7.0	50.0	1.020	4.0	0.0	NaN	normal	
2	62.0	80.0	1.010	2.0	3.0	normal	normal	
3	48.0	70.0	1.005	4.0	0.0	normal	abnormal	
4	51.0	80.0	1.010	2.0	0.0	normal	normal	
5	60.0	90.0	1.015	3.0	0.0	NaN	NaN	
6	68.0	70.0	1.010	0.0	0.0	NaN	normal	
7	24.0	NaN	1.015	2.0	4.0	normal	abnormal	
8	52.0	100.0	1.015	3.0	0.0	normal	abnormal	
9	53.0	90.0	1.020	2.0	0.0	abnormal	abnormal	

10 rows × 25 columns

In [19]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 25 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   age                       391 non-null    float64
 1   blood_pressure            388 non-null    float64
 2   specific_gravity          353 non-null    float64
 3   albumin                   354 non-null    float64
 4   sugar                     351 non-null    float64
 5   red_blood_cells           248 non-null    object 
 6   pus_cell                  335 non-null    object 
 7   pus_cell_clumps           396 non-null    object 
 8   bacteria                  396 non-null    object 
 9   blood_glucose_random      356 non-null    float64
 10  blood_urea                381 non-null    float64
 11  serum_creatinine          383 non-null    float64
 12  sodium                    313 non-null    float64
 13  potassium                 312 non-null    float64
 14  haemoglobin               348 non-null    float64
 15  packed_cell_volume        329 non-null    float64
 16  white_blood_cell_count    294 non-null    float64
 17  red_blood_cell_count      269 non-null    float64
 18  hypertension              398 non-null    object 
 19  diabetes_mellitus         398 non-null    object 
 20  coronary_artery_disease   398 non-null    object 
 21  appetite                  399 non-null    object 
 22  peda_edema                399 non-null    object 
 23  anemia                    399 non-null    object 
 24  class                     400 non-null    int64  
dtypes: float64(14), int64(1), object(10)
memory usage: 78.3+ KB
```

In [20]:
```
# checking for null values
df.isna().sum().sort_values(ascending = False)
```

Out[20]:
```
red_blood_cells            152
red_blood_cell_count       131
white_blood_cell_count     106
potassium                   88
sodium                      87
packed_cell_volume          71
pus_cell                    65
haemoglobin                 52
sugar                       49
specific_gravity            47
albumin                     46
blood_glucose_random        44
blood_urea                  19
serum_creatinine            17
blood_pressure              12
age                          9
bacteria                     4
pus_cell_clumps              4
hypertension                 2
diabetes_mellitus            2
coronary_artery_disease      2
appetite                     1
peda_edema                   1
anemia                       1
class                        0
dtype: int64
```

In [21]:
```
df[num_cols].isnull().sum().sort_values(ascending = False)
```

Out[21]:
```
red_blood_cell_count      131
white_blood_cell_count    106
potassium                  88
sodium                     87
packed_cell_volume         71
haemoglobin                52
sugar                      49
specific_gravity           47
albumin                    46
blood_glucose_random       44
blood_urea                 19
serum_creatinine           17
blood_pressure             12
age                         9
class                       0
dtype: int64
```

In [22]:
```
df[cat_cols].isnull().sum().sort_values(ascending = False)
```

Out[22]:
```
red_blood_cells            152
pus_cell                    65
pus_cell_clumps              4
bacteria                     4
hypertension                 2
diabetes_mellitus            2
coronary_artery_disease      2
appetite                     1
peda_edema                   1
anemia                       1
dtype: int64
```

In [23]:
```
# filling null values, we will use two methods, random sampling for hig
def random_value_imputation(feature):
    random_sample = df[feature].dropna().sample(df[feature].isna().sum(
    random_sample.index = df[df[feature].isnull()].index
    df.loc[df[feature].isnull(), feature] = random_sample

def impute_mode(feature):
    mode = df[feature].mode()[0]
    df[feature] = df[feature].fillna(mode)
```

In [24]:
```
# filling num_cols null values using random sampling method
for col in num_cols:
    random_value_imputation(col)

df[num_cols].isnull().sum()
```

Out[24]:
```
age                        0
blood_pressure             0
specific_gravity           0
albumin                    0
sugar                      0
blood_glucose_random       0
blood_urea                 0
serum_creatinine           0
sodium                     0
potassium                  0
haemoglobin                0
packed_cell_volume         0
white_blood_cell_count     0
red_blood_cell_count       0
class                      0
dtype: int64
```

In [25]:
```
# filling "red_blood_cells" and "pus_cell" using random sampling metho
random_value_imputation('red_blood_cells')
random_value_imputation('pus_cell')

for col in cat_cols:
    impute_mode(col)

df[cat_cols].isnull().sum()
```

Out[25]:
```
red_blood_cells            0
pus_cell                   0
pus_cell_clumps            0
bacteria                   0
hypertension               0
diabetes_mellitus          0
coronary_artery_disease    0
appetite                   0
peda_edema                 0
anemia                     0
dtype: int64
```

In [26]:
```
#Feature Encoding
```

In [27]:
```
# Printing unique values in cat_cols
for col in cat_cols:
    print(f"{col} has {df[col].nunique()} categories\n")
```

```
red_blood_cells has 2 categories

pus_cell has 2 categories

pus_cell_clumps has 2 categories

bacteria has 2 categories

hypertension has 2 categories

diabetes_mellitus has 2 categories

coronary_artery_disease has 2 categories

appetite has 2 categories

peda_edema has 2 categories

anemia has 2 categories
```

In [28]:
```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for col in cat_cols:
    df[col] = le.fit_transform(df[col])
```

In [29]:
```
df.head(10)
```

Out[29]:

	age	blood_pressure	specific_gravity	albumin	sugar	red_blood_cells	pus_cell	pu
0	48.0	80.0	1.020	1.0	0.0	1	1	
1	7.0	50.0	1.020	4.0	0.0	1	1	
2	62.0	80.0	1.010	2.0	3.0	1	1	
3	48.0	70.0	1.005	4.0	0.0	1	0	
4	51.0	80.0	1.010	2.0	0.0	1	1	
5	60.0	90.0	1.015	3.0	0.0	1	1	
6	68.0	70.0	1.010	0.0	0.0	1	1	
7	24.0	100.0	1.015	2.0	4.0	1	0	
8	52.0	100.0	1.015	3.0	0.0	1	0	
9	53.0	90.0	1.020	2.0	0.0	0	0	

10 rows × 25 columns

In [30]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 25 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   age                      400 non-null    float64
 1   blood_pressure           400 non-null    float64
 2   specific_gravity         400 non-null    float64
 3   albumin                  400 non-null    float64
 4   sugar                    400 non-null    float64
 5   red_blood_cells          400 non-null    int32  
 6   pus_cell                 400 non-null    int32  
 7   pus_cell_clumps          400 non-null    int32  
 8   bacteria                 400 non-null    int32  
 9   blood_glucose_random     400 non-null    float64
 10  blood_urea               400 non-null    float64
 11  serum_creatinine         400 non-null    float64
 12  sodium                   400 non-null    float64
 13  potassium                400 non-null    float64
 14  haemoglobin              400 non-null    float64
 15  packed_cell_volume       400 non-null    float64
 16  white_blood_cell_count   400 non-null    float64
 17  red_blood_cell_count     400 non-null    float64
 18  hypertension             400 non-null    int32  
 19  diabetes_mellitus        400 non-null    int32  
 20  coronary_artery_disease  400 non-null    int32  
 21  appetite                 400 non-null    int32  
 22  peda_edema               400 non-null    int32  
 23  anemia                   400 non-null    int32  
 24  class                    400 non-null    int64  
dtypes: float64(14), int32(10), int64(1)
memory usage: 62.6 KB
```

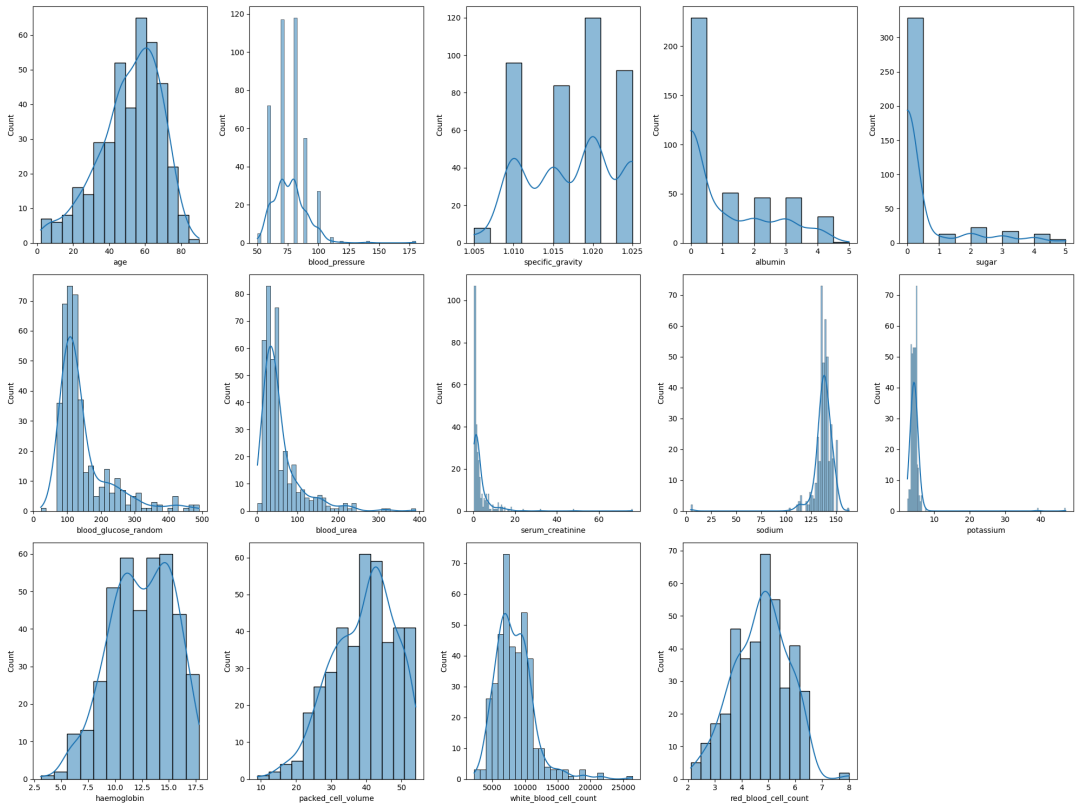
In [31]:

```
# checking numerical features distribution
Num_fea_path = os.path.join(graphs_dir, 'numerical_feature.png')
plt.figure(figsize = (20, 15))
plotnumber = 1

for column in num_cols:
    if plotnumber <= 14:
        ax = plt.subplot(3, 5, plotnumber)
        sns.histplot(df[column], kde=True)
        plt.xlabel(column)

    plotnumber += 1

plt.tight_layout()
plt.savefig(Num_fea_path)
plt.show()
```



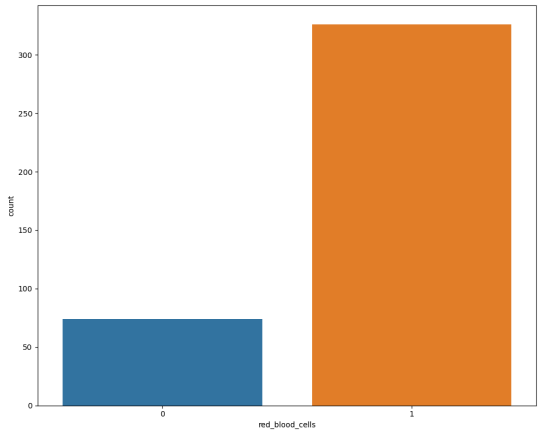
In [32]:

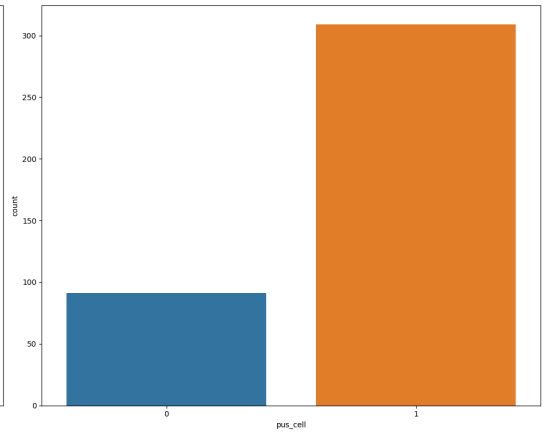
```
# Checking categorical columns
cat_col_path = os.path.join(graphs_dir, 'categorical_columns.png')
plt.figure(figsize=(20, 40))
plotnumber = 1

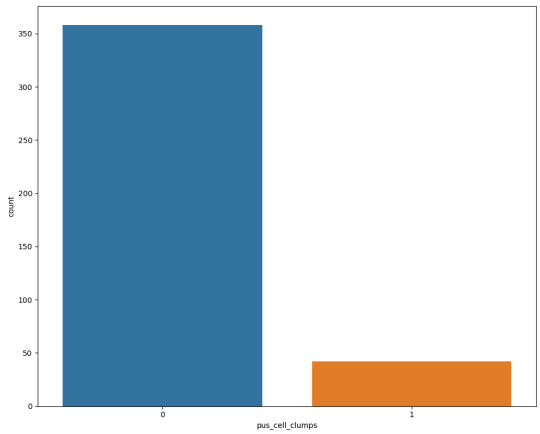
for column in cat_cols:
    if plotnumber <= 11:
        ax = plt.subplot(5, 2, plotnumber)
        sns.countplot(data=df, x=column)
        plt.xlabel(column)

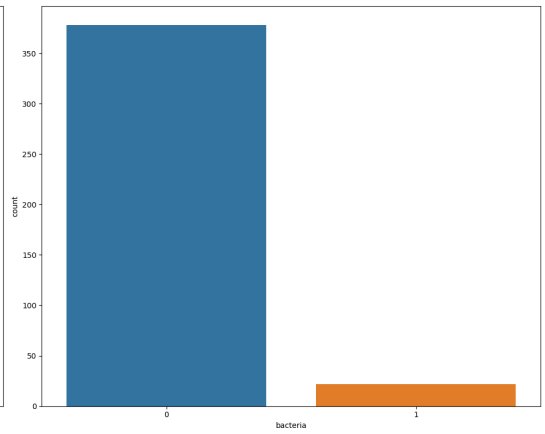
    plotnumber += 1

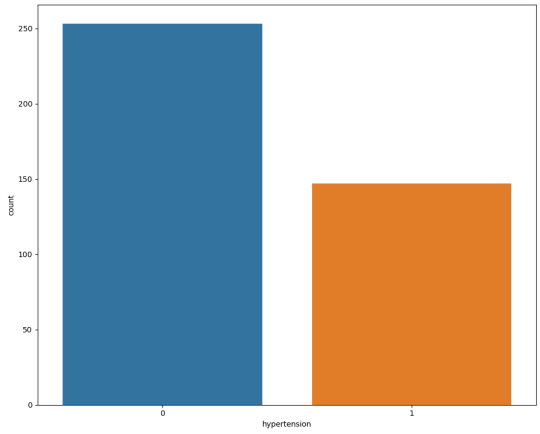
plt.tight_layout()
plt.savefig(cat_col_path)
plt.show()
```

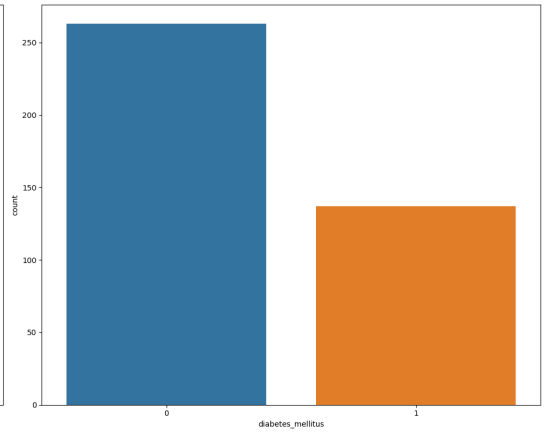

count
red_blood_cells
0
1

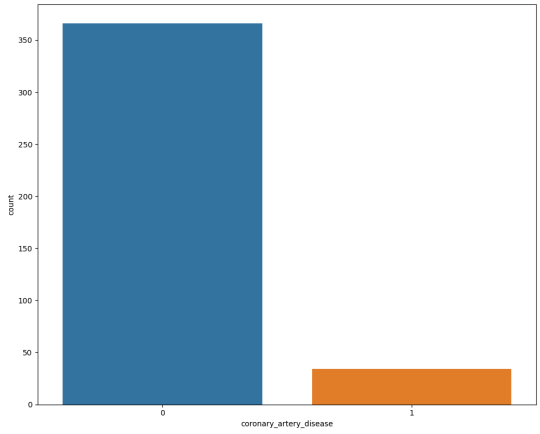

count
pus_cell
0
1

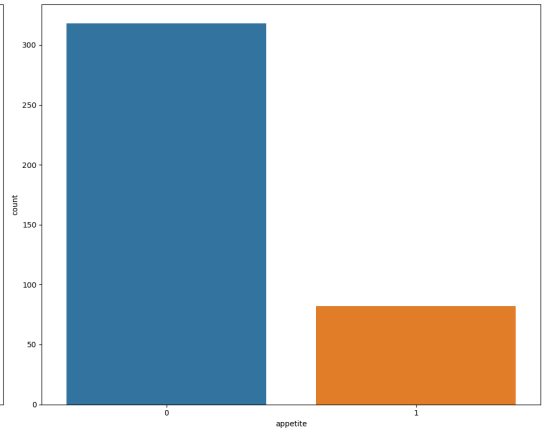

count
pus_cell_clumps
0
1

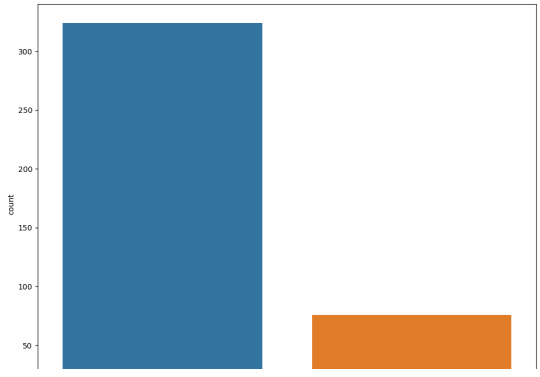

count
bacteria
0
1

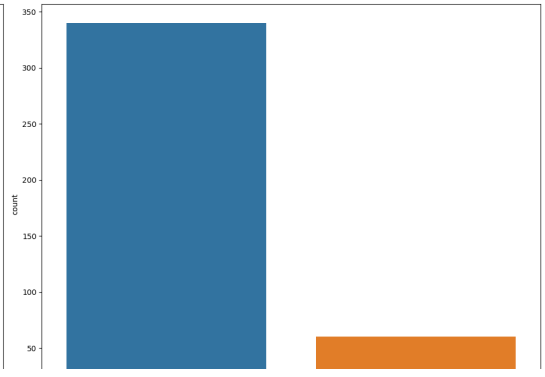

count
hypertension
0
1


count
diabetes_mellitus
0
1


count
coronary_artery_disease
0
1


count
appetite
0
1

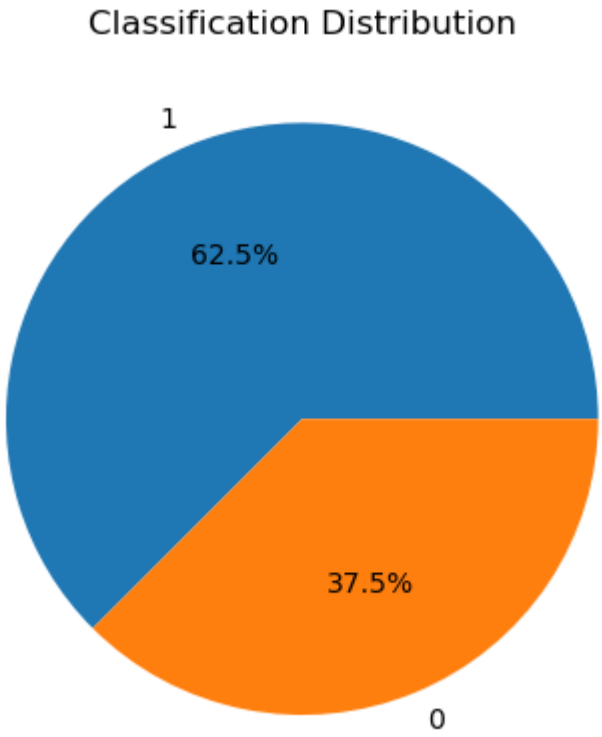

count


count

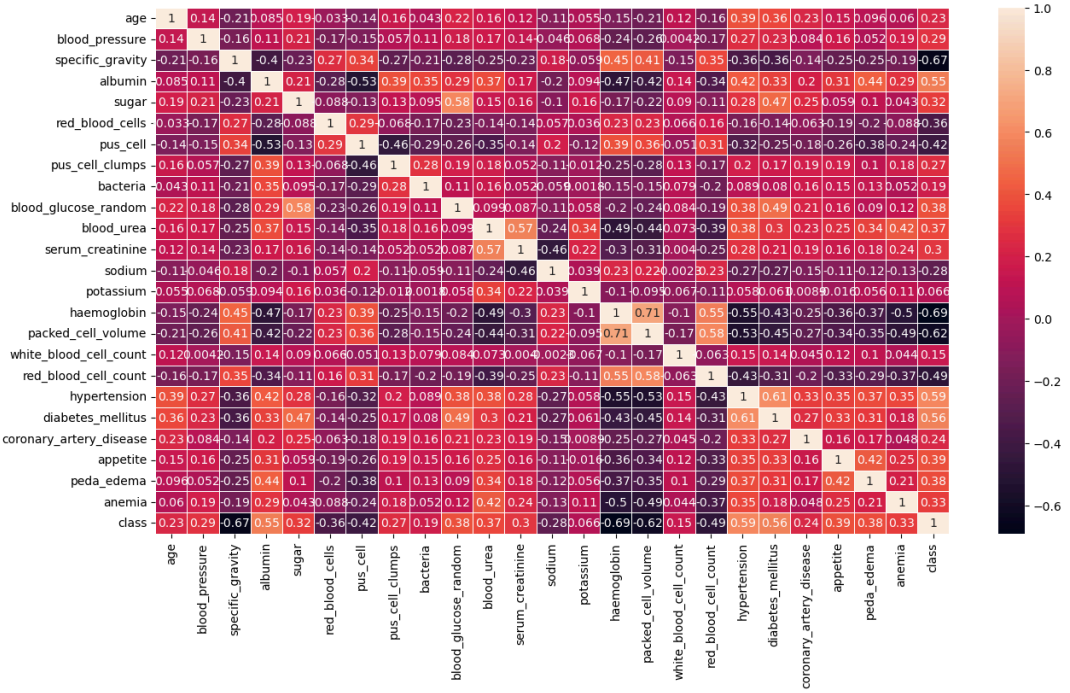


In [33]:

```
# Checking target class
targetclass_path=os.path.join(graphs_dir, 'target_class.png')
plt.pie(df['class'].value_counts(), labels=df['class'].value_counts().i
plt.title('Classification Distribution')
plt.savefig(targetclass_path)
plt.show()
```



In [34]:
```
# heatmap of the data
heatmap_path=os.path.join(graphs_dir, 'heatmap.png')
plt.figure(figsize = (15, 8))
sns.heatmap(df.corr(), annot = True, linewidths = 0.5)
plt.savefig(heatmap_path)
plt.show()
```

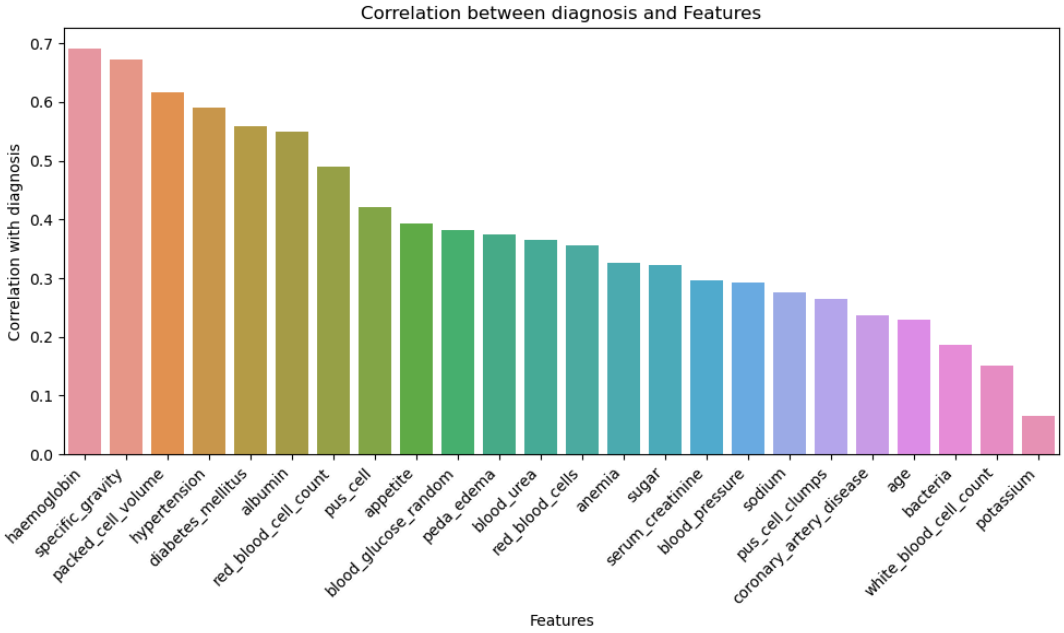


In [35]:
```
# Calculate the correlation list
target_corr = df.corr()['class'].abs().sort_values(ascending=False)[1:]
print(target_corr)
```

```
haemoglobin                0.691336
specific_gravity           0.672359
packed_cell_volume         0.615740
hypertension               0.590438
diabetes_mellitus          0.559060
albumin                    0.550244
red_blood_cell_count       0.490463
pus_cell                   0.420356
appetite                   0.393341
blood_glucose_random       0.381666
peda_edema                 0.375154
blood_urea                 0.365025
red_blood_cells            0.355749
anemia                     0.325396
sugar                      0.322741
serum_creatinine           0.295764
blood_pressure             0.292424
sodium                     0.275007
pus_cell_clumps            0.265313
coronary_artery_disease    0.236088
age                        0.229951
bacteria                   0.186871
white_blood_cell_count     0.150237
potassium                  0.065840
Name: class, dtype: float64
```

In [36]:

```
# Create a bar chart to visualize the correlations
correlations_path=os.path.join(graphs_dir, 'correlation.png')
plt.figure(figsize=(10, 6))
sns.barplot(x=target_corr.index, y=target_corr.values)
plt.xticks(rotation=45, ha='right')
plt.xlabel('Features')
plt.ylabel('Correlation with diagnosis')
plt.title('Correlation between diagnosis and Features')
plt.tight_layout()
plt.savefig(correlations_path)
plt.show()
```



In [38]:

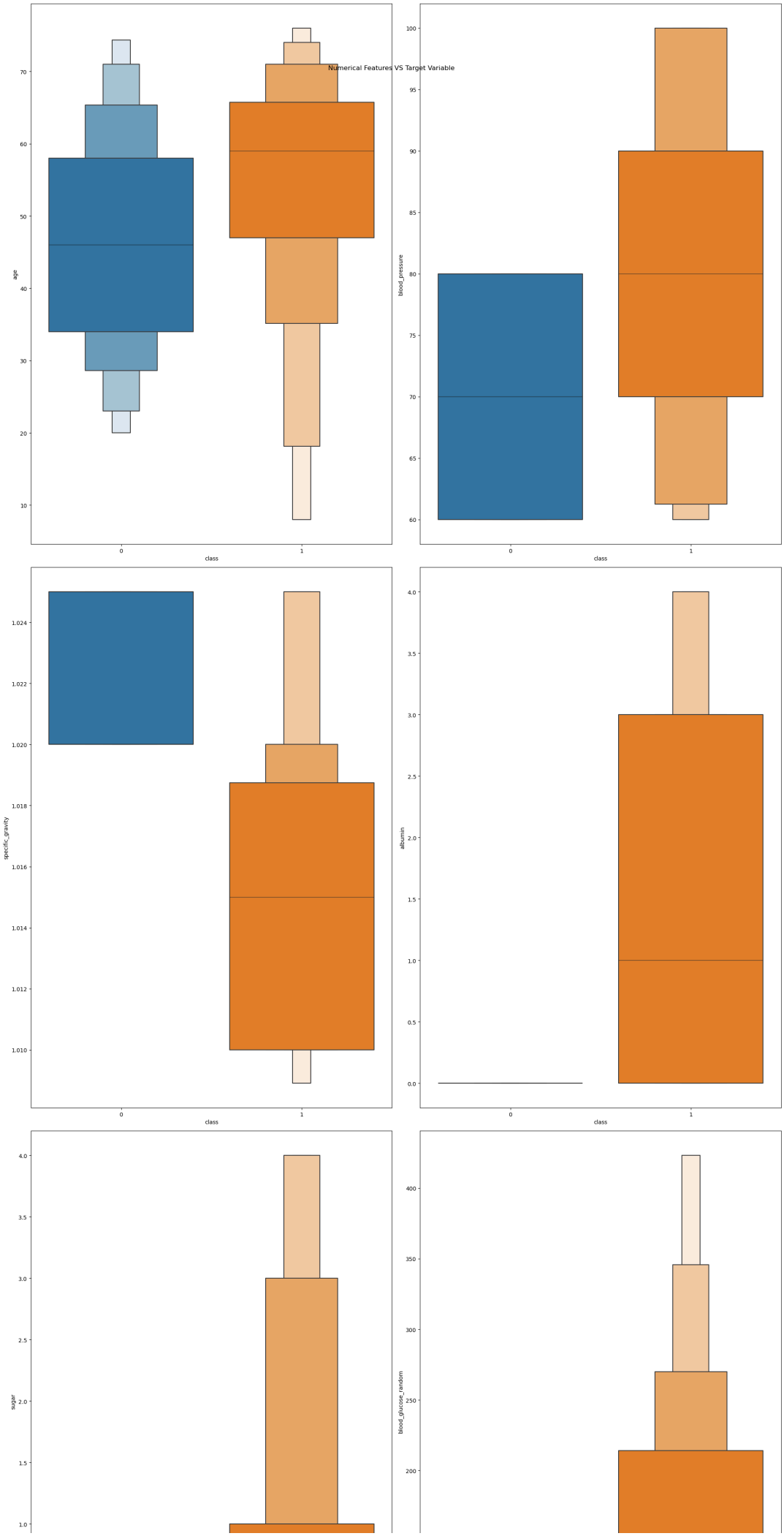
```
# Creating a boxen plot to show features against the target
n_rows, n_cols = (7,2)

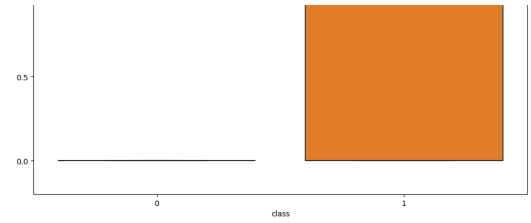
boxen_plot = os.path.join(graphs_dir, 'boxen_plot.png')

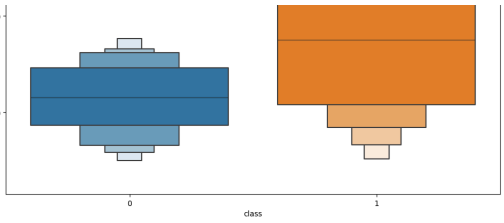
figure, axes = plt.subplots(nrows=n_rows, ncols=n_cols, figsize = (20,
figure.suptitle('Numerical Features VS Target Variable')

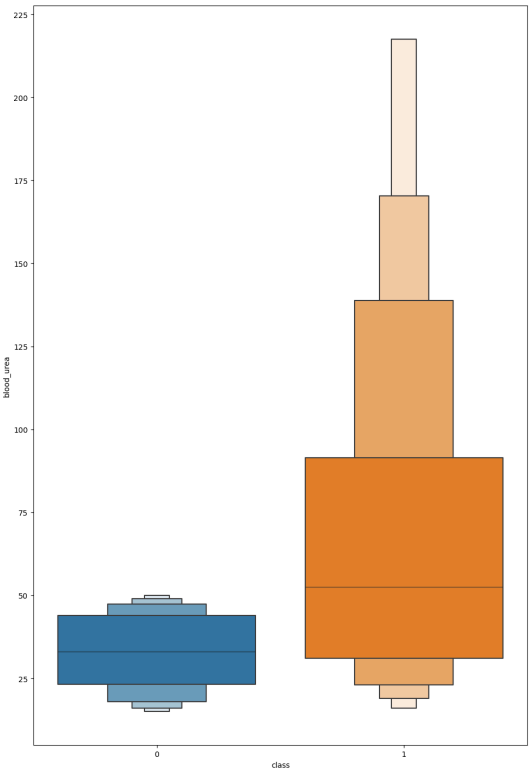
for index, column in enumerate(num_cols[:-1]):

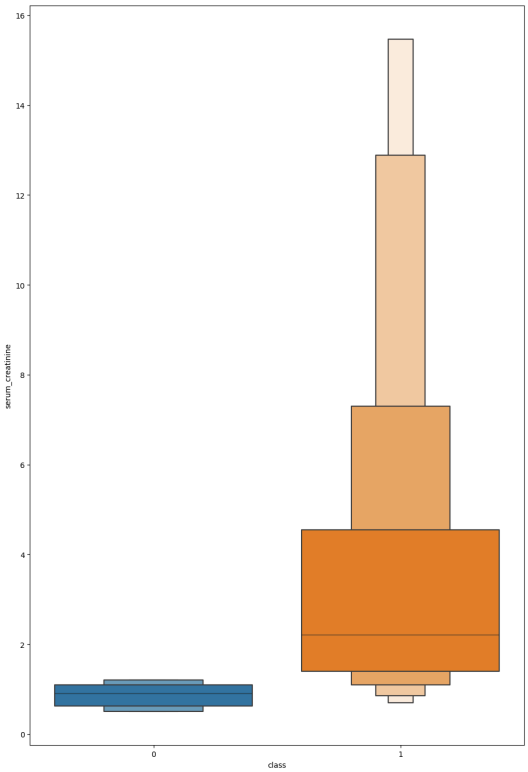
    i,j = (index // n_cols), (index % n_cols)
    bp=sns.boxenplot(y=column, x='class', data=df, ax=axes[i,j], showfl
    axes[i,j].set_xlabel(axes[i,j].get_xlabel())
    axes[i,j].set_ylabel(column)
    axes[i,j].set_xticklabels(axes[i,j].get_xticklabels())
plt.tight_layout()
plt.savefig(boxen_plot)
plt.show()
```

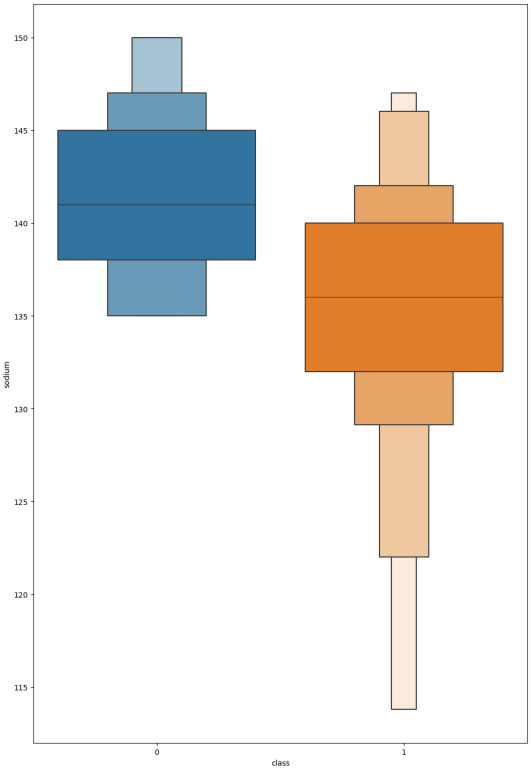

Numerical Features VS Target Variable
age
blood_pressure
specific_gravity
albumin
sugar
blood_glucose_random
class

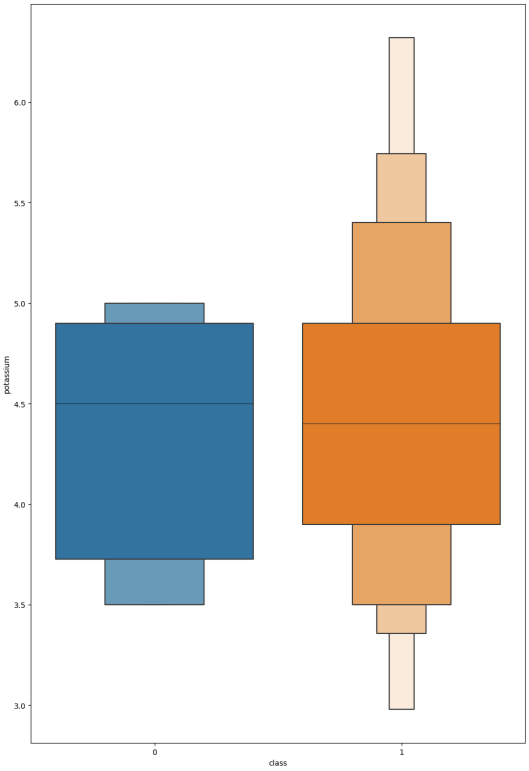

0.5
0.0
0
1
class

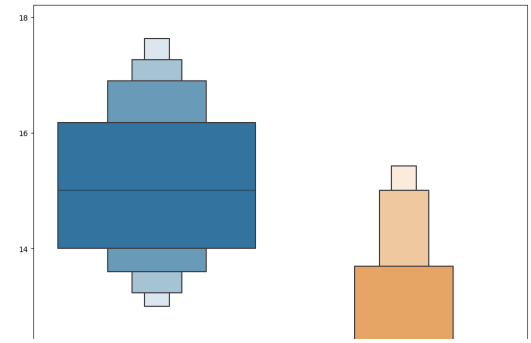

150
100
0
1
class

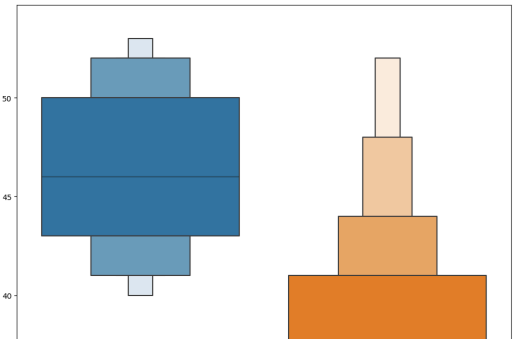

blood_urea
225
200
175
150
125
100
75
50
25
0
1
class

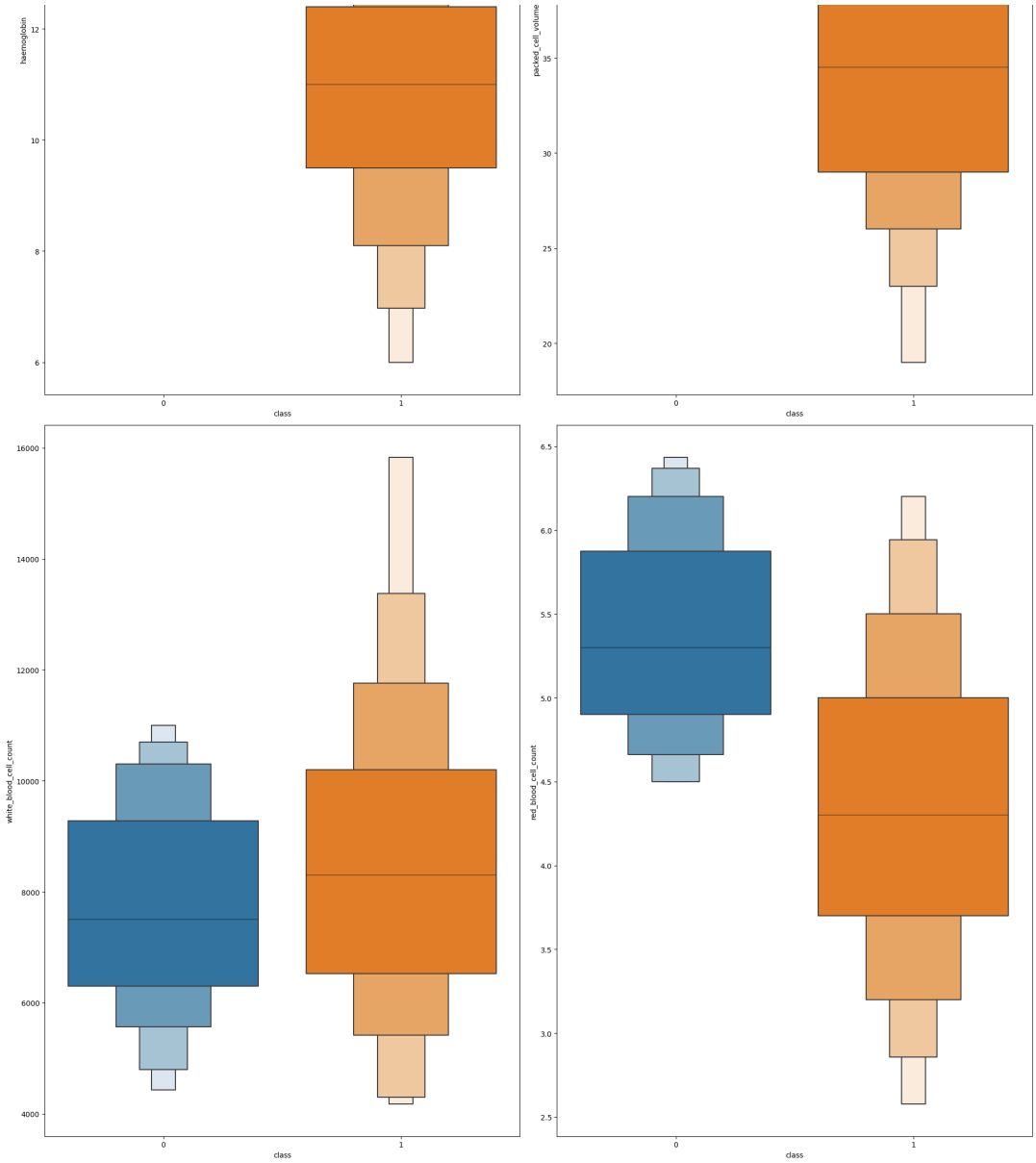

serum_creatinine
16
14
12
10
8
6
4
2
0
0
1
class


sodium
150
145
140
135
130
125
120
115
0
1
class


potassium
6.0
5.5
5.0
4.5
4.0
3.5
3.0
0
1
class


18
16
14

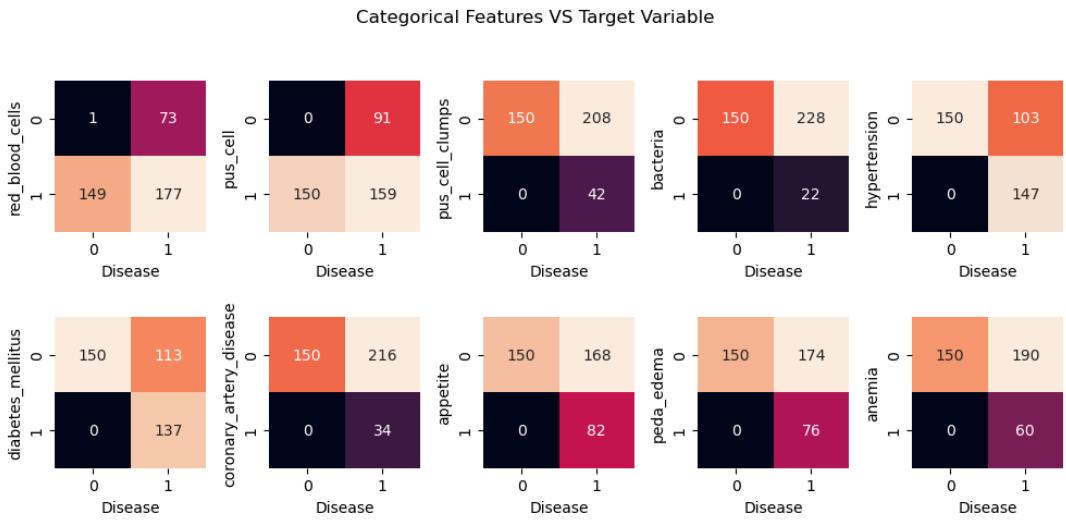

50
45
40


haemoglobin
packed_cell_volume
white_blood_cell_count
red_blood_cell_count
class
0
1

In [39]:
```
n_rows, n_cols = (2,5)
figure, axes = plt.subplots(nrows=n_rows, ncols=n_cols,figsize=(10, 5))
figure.suptitle('Categorical Features VS Target Variable')
cat_vs_tar_var= os.path.join(graphs_dir, 'cat cols vs target var.png')
for index, column in enumerate(cat_cols):
    i,j = (index // n_cols), (index % n_cols)
    sns.heatmap(pd.crosstab(df[column], df['class']), ax=axes[i,j], squ

    axes[i,j].set_xlabel("Disease")
    axes[i,j].set_ylabel(column)

plt.tight_layout()
plt.savefig(cat_vs_tar_var)
plt.show()
```



In [40]:
```
#Modelling
```

In [41]:
```
base_dir=r"C:\Users\Tharunraj_S\Desktop\Mini_project\Codes\ML logs\mode
```

In [57]:

```
from sklearn.metrics import accuracy_score, roc_auc_score

def print_score(clf, X_train, y_train, X_test, y_test, y_train_prob, y_
    pred = None
    if train:
        pred = clf.predict(X_train)
        train_accuracy = accuracy_score(y_train, pred) * 100
        train_roc_auc = roc_auc_score(y_train, y_train_prob) * 100

        train_results = {
            "Accuracy Score": train_accuracy,
            "ROC AUC Score": train_roc_auc,
            "Confusion Matrix": confusion_matrix(y_train, pred).tolist(
        }
        save_results_to_csv(train_results, "Train_Results.csv")

        print("Train Result:\n==========================================
        print(f"Accuracy Score: {train_accuracy:.2f}%")
        print("_____________________________________________")
        print(f"ROC AUC Score: {train_roc_auc:.2f}%")
        print("_____________________________________________")
        print(f"CLASSIFICATION REPORT:\n{classification_report(y_train,
        print("_____________________________________________")
        print(f"Confusion Matrix: \n {train_results['Confusion Matrix']
        plt.figure(figsize=(10, 8))
        sns.heatmap(train_results['Confusion Matrix'], annot=True, fmt=
        plt.xlabel('Predicted Label')
        plt.ylabel('True Label')
        plt.title('Confusion Matrix')
        plt.savefig('Confusion Matrix Train set')
        plt.show()
        return train_accuracy, train_roc_auc

    elif not train:
        pred = clf.predict(X_test)
        test_accuracy = accuracy_score(y_test, pred) * 100
        test_roc_auc = roc_auc_score(y_test, y_test_prob) * 100

        test_results = {
            "Accuracy Score": test_accuracy,
            "ROC AUC Score": test_roc_auc,
            "Confusion Matrix": confusion_matrix(y_test, pred).tolist()
        }
        save_results_to_csv(test_results, "Test_Results.csv")

        print("Test Result:\n==========================================
        print(f"Accuracy Score: {test_accuracy:.2f}%")
        print("_____________________________________________")
        print(f"ROC AUC Score: {test_roc_auc:.2f}%")
        print("_____________________________________________")
        print(f"CLASSIFICATION REPORT:\n{classification_report(y_test,
        print("_____________________________________________")
        print(f"Confusion Matrix: \n {test_results['Confusion Matrix']}
        plt.figure(figsize=(10, 8))
        sns.heatmap(test_results['Confusion Matrix'], annot=True, fmt='
        plt.xlabel('Predicted Label')
        plt.ylabel('True Label')
        plt.title('Confusion Matrix')
        plt.savefig('Confusion Matrix Test set')
        plt.show()
        return test_accuracy, test_roc_auc
```

In [58]:
```
def create_dir(subdir):
    if not os.path.exists(subdir):
        os.makedirs(os.path.join(base_dir, subdir), exist_ok=True)
    os.chdir(os.path.join(base_dir, subdir))
    updated_dir = os.getcwd()
    print("Saving in directory:", updated_dir)

def save_results_to_csv(results, filename):
    df = pd.DataFrame(results)
    df.to_csv(filename, index=False)
```

In [59]:
```
#SPLITTING AND SCALING
```

In [60]:
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.model_selection import train_test_split, GridSearchCV


X = df.drop("class", axis=1)
y = df["class"]

print(f"'X' shape: {X.shape}")
print(f"'y' shape: {y.shape}")

pipeline = Pipeline([
    ('min_max_scaler', MinMaxScaler()),
    ('std_scaler', StandardScaler())])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2
X_train = pipeline.fit_transform(X_train)
X_test = pipeline.transform(X_test)
```

```
'X' shape: (400, 24)
'y' shape: (400,)
```

In [61]:
```
#LOGISTIC REGRESSION
```

In [63]:
```
from scipy.stats import loguniform
from sklearn.linear_model import LogisticRegression

os.chdir(base_dir)
create_dir('Logistic Regression')


param_grid = dict()
param_grid['solver'] = ['newton-cg', 'lbfgs', 'liblinear']
param_grid['penalty'] = ['l2'] #'none', 'l1', 'l2', 'elasticnet'
param_grid['C'] = loguniform.rvs(1e-5, 100, size=10)

grid = GridSearchCV(LogisticRegression(), param_grid, refit=True, verbo
model = grid.fit(X_train, y_train).best_estimator_

best_params = grid.best_params_
print(f"Best params: {best_params}")

y_train_prob = model.predict_proba(X_train)[:, 1]
y_test_prob = model.predict_proba(X_test)[:, 1]

print_score(model, X_train, y_train, X_test, y_test, y_train_prob, y_te
lr_acc, lr_ra = print_score(model, X_train, y_train, X_test, y_test, y_
joblib.dump(model,'logistic_regression.pkl')
```

```
Saving in directory: C:\Users\Tharunraj_S\Desktop\Mini_project\Codes\M
L logs\models\Logistic Regression
Fitting 5 folds for each of 30 candidates, totalling 150 fits
Best params: {'C': 3.403785256882984, 'penalty': 'l2', 'solver': 'newt
on-cg'}
Train Result:
================================================
Accuracy Score: 100.00%

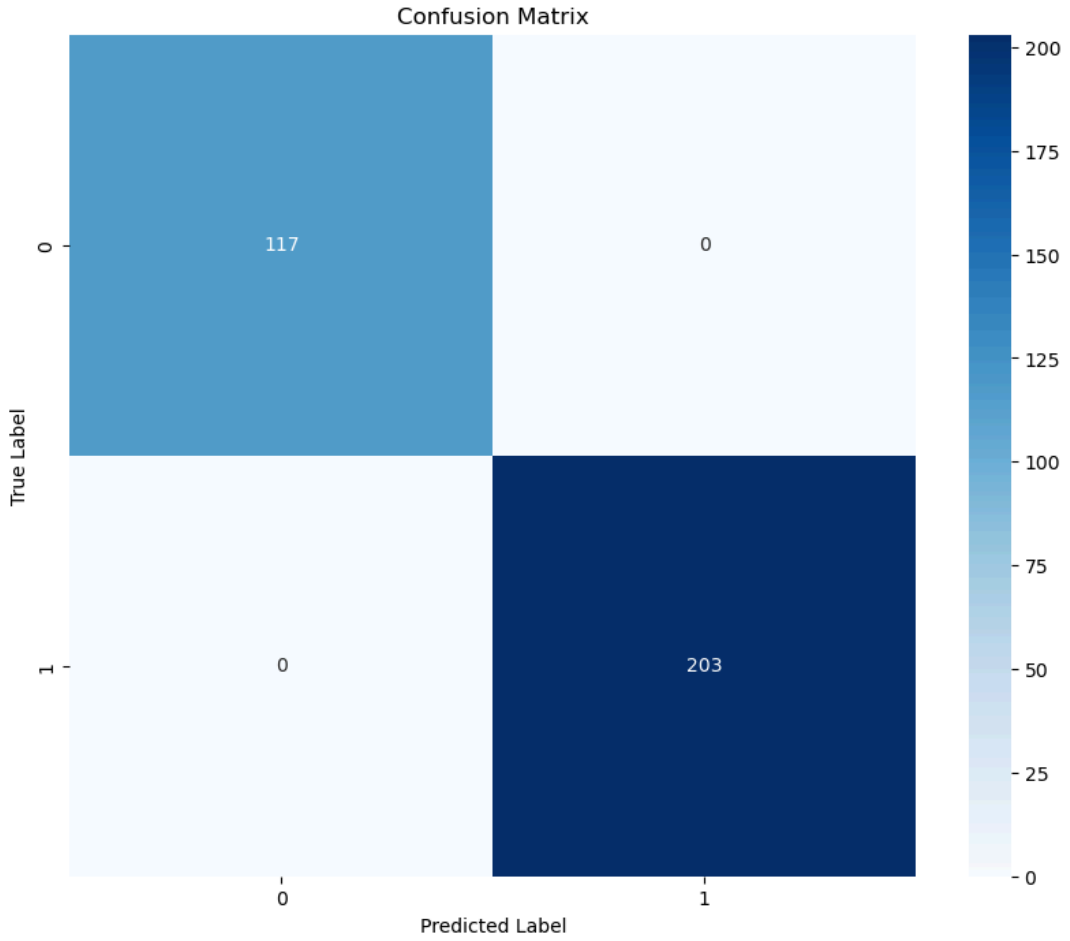
_______________________________________________
ROC AUC Score: 100.00%

_______________________________________________
CLASSIFICATION REPORT:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       117
           1       1.00      1.00      1.00       203

    accuracy                           1.00       320
   macro avg       1.00      1.00      1.00       320
weighted avg       1.00      1.00      1.00       320


_______________________________________________
Confusion Matrix:
 [[117, 0], [0, 203]]
```

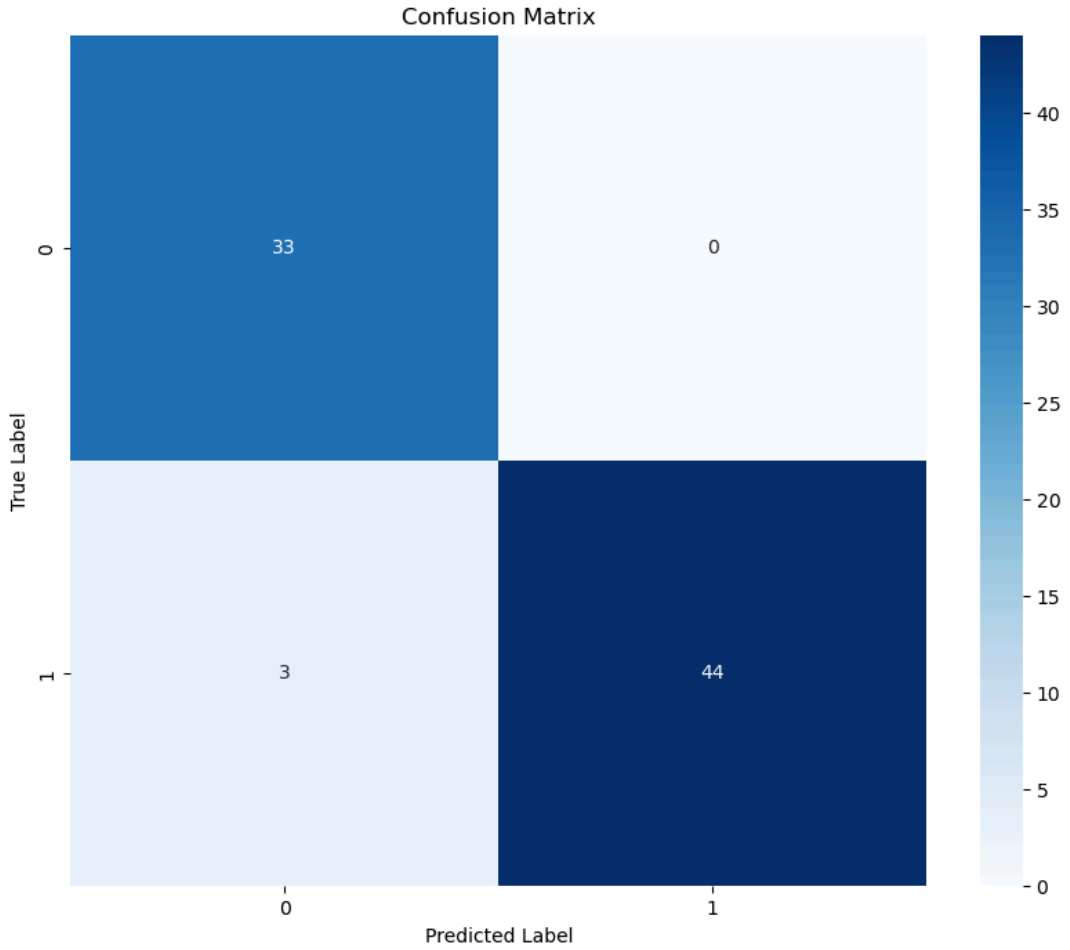



```
Test Result:
================================================
Accuracy Score: 96.25%
_________________________________________________
ROC AUC Score: 99.87%
_________________________________________________
CLASSIFICATION REPORT:
              precision    recall  f1-score   support

           0       0.92      1.00      0.96        33
           1       1.00      0.94      0.97        47

    accuracy                           0.96        80
   macro avg       0.96      0.97      0.96        80
weighted avg       0.97      0.96      0.96        80


_________________________________________________
Confusion Matrix:
 [[33, 0], [3, 44]]
```



Out[63]: ['logistic_regression.pkl']

In [49]:
```
#KNN
```

In [64]:

```
from sklearn.neighbors import KNeighborsClassifier

os.chdir(base_dir)
create_dir('KNeighborsClassifier')

param_grid = {"n_neighbors": [i for i in range(1, 50, 10)],
              "weights": ["uniform", "distance"],
              "algorithm": ["ball_tree", "kd_tree", "brute"],
              "leaf_size": [i for i in range(1, 50, 10)],
              "p": [1,2]}

grid = GridSearchCV(KNeighborsClassifier(), param_grid, refit=True, ver
model = grid.fit(X_train, y_train).best_estimator_

best_params = grid.best_params_
print(f"Best params: {best_params}")

y_train_prob = model.predict_proba(X_train)[:, 1]
y_test_prob = model.predict_proba(X_test)[:, 1]

print_score(model, X_train, y_train, X_test, y_test, y_train_prob, y_te
knn_acc, knn_ra =print_score(model, X_train, y_train, X_test, y_test, y
joblib.dump(model,'KNeighborsClassifier.pkl')
```

```
Saving in directory: C:\Users\Tharunraj_S\Desktop\Mini_project\Codes\M
L logs\models\KNeighborsClassifier
Fitting 5 folds for each of 300 candidates, totalling 1500 fits
Best params: {'algorithm': 'ball_tree', 'leaf_size': 1, 'n_neighbors':
1, 'p': 2, 'weights': 'uniform'}
Train Result:
================================================
Accuracy Score: 100.00%

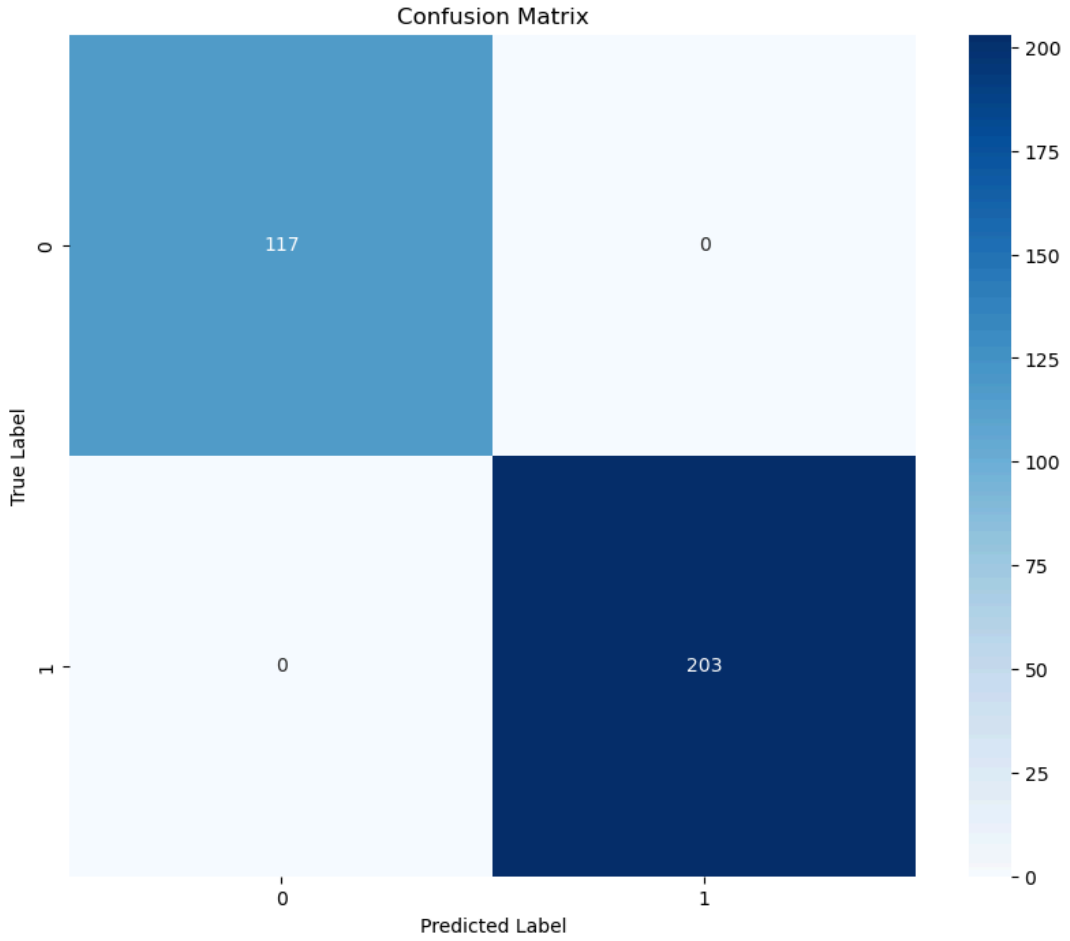
_______________________________________________
ROC AUC Score: 100.00%

_______________________________________________
CLASSIFICATION REPORT:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       117
           1       1.00      1.00      1.00       203

    accuracy                           1.00       320
   macro avg       1.00      1.00      1.00       320
weighted avg       1.00      1.00      1.00       320


_______________________________________________
Confusion Matrix:
 [[117, 0], [0, 203]]
```




```
Test Result:
================================================
Accuracy Score: 95.00%

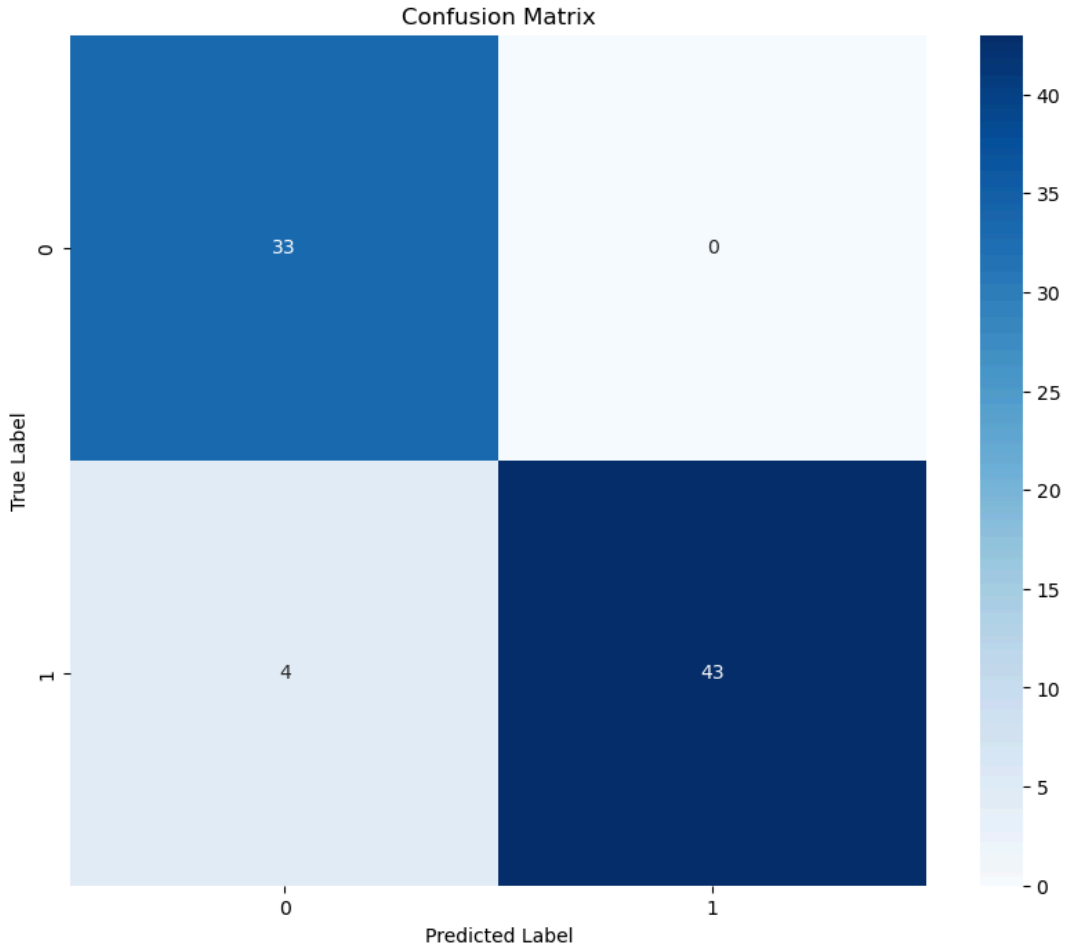
_________________________________________________
ROC AUC Score: 95.74%

_________________________________________________
CLASSIFICATION REPORT:
              precision    recall  f1-score   support

           0       0.89      1.00      0.94        33
           1       1.00      0.91      0.96        47

    accuracy                           0.95        80
   macro avg       0.95      0.96      0.95        80
weighted avg       0.96      0.95      0.95        80


_________________________________________________
Confusion Matrix:
 [[33, 0], [4, 43]]
```



Out[64]: ['KNeighborsClassifier.pkl']

In [51]:
```
#DECISION TREE
```

In [65]:

```
from sklearn import tree
from sklearn.tree import DecisionTreeClassifier

os.chdir(base_dir)
create_dir('DecisionTreeClassifier')

param_grid = {"max_depth": [3, 5, 7, 10, 15, 20, None],
              "max_features": [None, 'sqrt', 'log2'],
              "min_samples_leaf": [1, 3, 5, 10, 20],
              "criterion": ["gini", "entropy"]}

grid = GridSearchCV(DecisionTreeClassifier(), param_grid, refit=True, v
model = grid.fit(X_train, y_train).best_estimator_

best_params = grid.best_params_
print(f"Best params: {best_params}")

y_train_prob = model.predict_proba(X_train)[:, 1]
y_test_prob = model.predict_proba(X_test)[:, 1]

print_score(model, X_train, y_train, X_test, y_test, y_train_prob, y_te
dtc_acc, dtc_ra =print_score(model, X_train, y_train, X_test, y_test, y
plt.figure(figsize=(20, 18))
tree.plot_tree(model, feature_names = df.columns.tolist()[1:], filled=T
plt.savefig("DECISION TREE")
plt.show()
joblib.dump(model,'DecisionTreeClassifier.pkl')
```

```
Saving in directory: C:\Users\Tharunraj_S\Desktop\Mini_project\Codes\M
L logs\models\DecisionTreeClassifier
Fitting 5 folds for each of 210 candidates, totalling 1050 fits
Best params: {'criterion': 'entropy', 'max_depth': 5, 'max_features':
'log2', 'min_samples_leaf': 5}
Train Result:
================================================
Accuracy Score: 97.81%

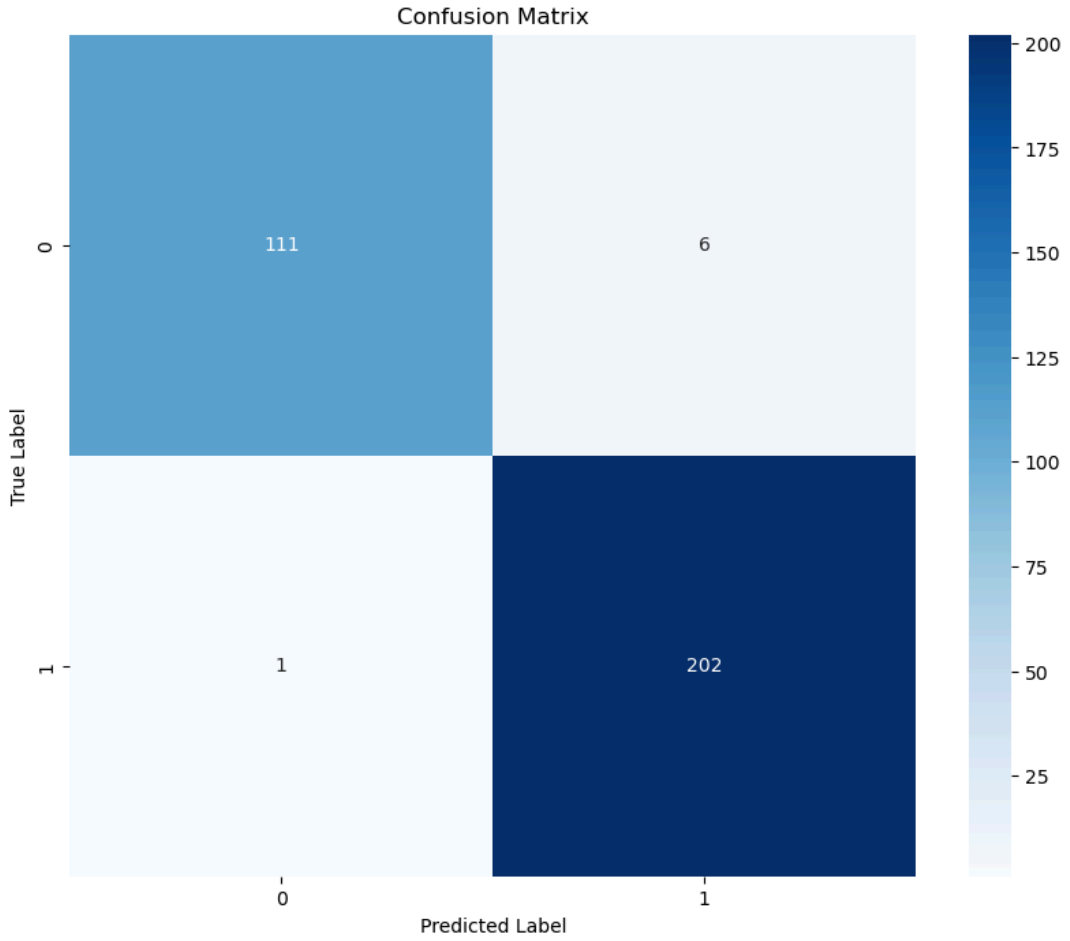
_______________________________________________
ROC AUC Score: 99.77%

_______________________________________________
CLASSIFICATION REPORT:
              precision    recall  f1-score   support

           0       0.99      0.95      0.97       117
           1       0.97      1.00      0.98       203

    accuracy                           0.98       320
   macro avg       0.98      0.97      0.98       320
weighted avg       0.98      0.98      0.98       320


_______________________________________________
Confusion Matrix:
 [[111, 6], [1, 202]]
```

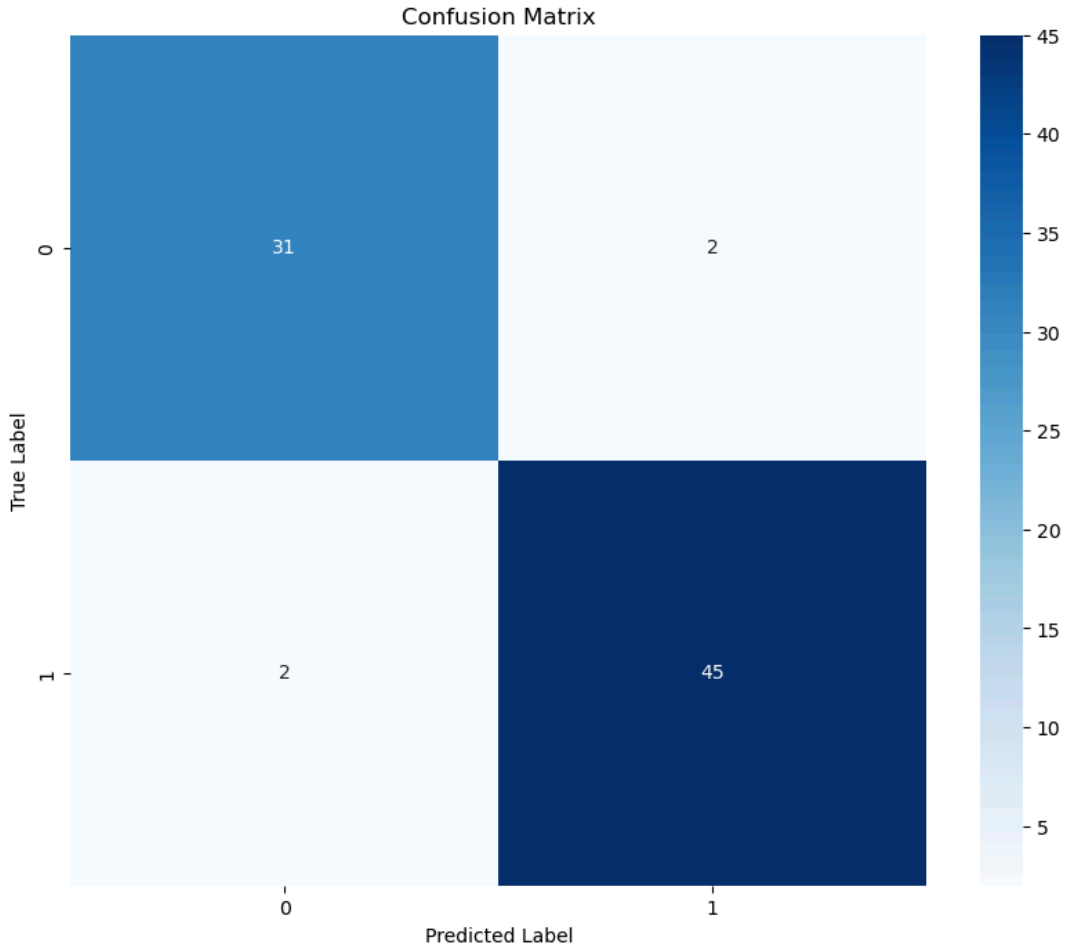


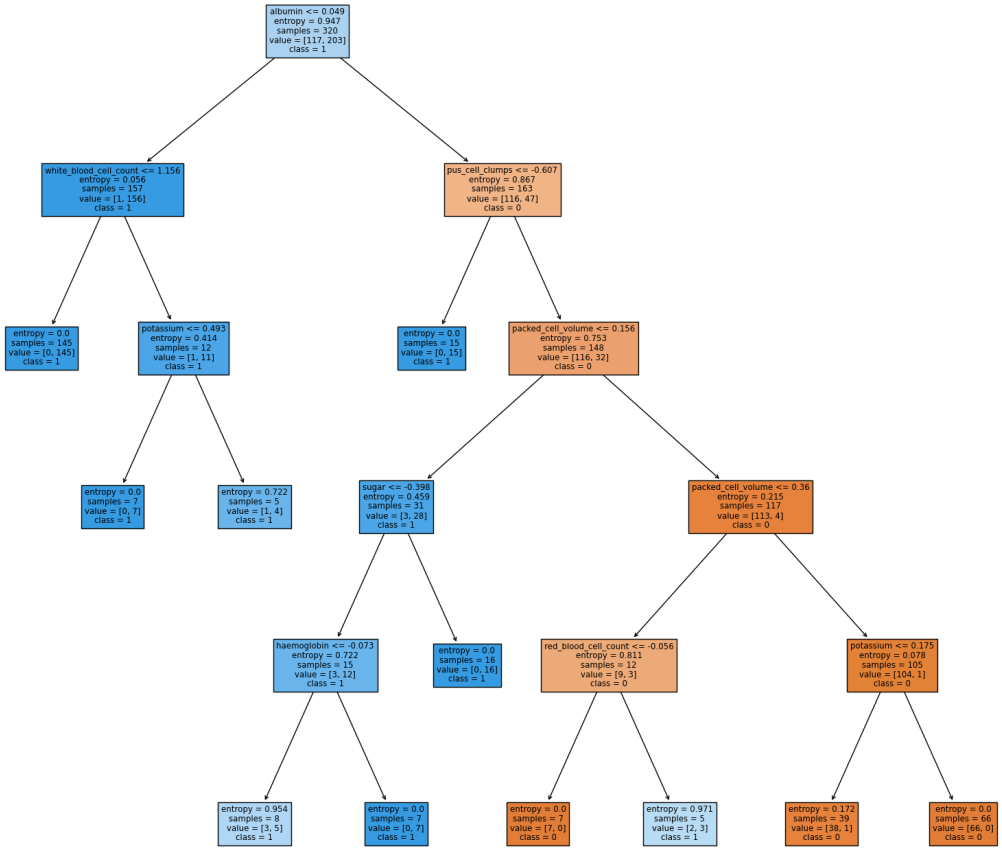

```
Test Result:
================================================
Accuracy Score: 95.00%
_________________________________________________
ROC AUC Score: 97.58%
_________________________________________________
CLASSIFICATION REPORT:
              precision    recall  f1-score   support

           0       0.94      0.94      0.94        33
           1       0.96      0.96      0.96        47

    accuracy                           0.95        80
   macro avg       0.95      0.95      0.95        80
weighted avg       0.95      0.95      0.95        80


_________________________________________________
Confusion Matrix:
 [[31, 2], [2, 45]]
```





Out[65]: ['DecisionTreeClassifier.pkl']

In [54]:
```
#RANDOM FOREST
X_train.shape
```

Out[54]:
```
(320, 24)
```

In [66]:

```
from sklearn.ensemble import RandomForestClassifier

os.chdir(base_dir)
create_dir('RandomForestClassifier')

param_grid = {'bootstrap': [True, False],
              'max_depth': [None, 5, 10, 15, 20],
              'max_features': [None, 'sqrt'],
              'min_samples_leaf': [1, 10, 20],
              'min_samples_split': [2, 10, 20],
              'n_estimators': [50, 100]}

grid = GridSearchCV(RandomForestClassifier(), param_grid, refit=True, v
model = grid.fit(X_train, y_train).best_estimator_

best_params = grid.best_params_
print(f"Best params: {best_params}")

y_train_prob = model.predict_proba(X_train)[:, 1]
y_test_prob = model.predict_proba(X_test)[:, 1]

print_score(model, X_train, y_train, X_test, y_test, y_train_prob, y_te
rd_clf_acc, rd_clf_ra =print_score(model, X_train, y_train, X_test, y_t
rfi=pd.Series(model.feature_importances_, index=df.columns.tolist()[:-1
plt.barh(rfi.index,rfi.values)
plt.savefig("RFC")
plt.show()
joblib.dump(model,'RandomForestClassifier.pkl')
```

```
Saving in directory: C:\Users\Tharunraj_S\Desktop\Mini_project\Codes\M
L logs\models\RandomForestClassifier
Fitting 5 folds for each of 360 candidates, totalling 1800 fits
Best params: {'bootstrap': False, 'max_depth': None, 'max_features':
'sqrt', 'min_samples_leaf': 20, 'min_samples_split': 20, 'n_estimator
s': 50}
Train Result:
================================================
Accuracy Score: 99.38%

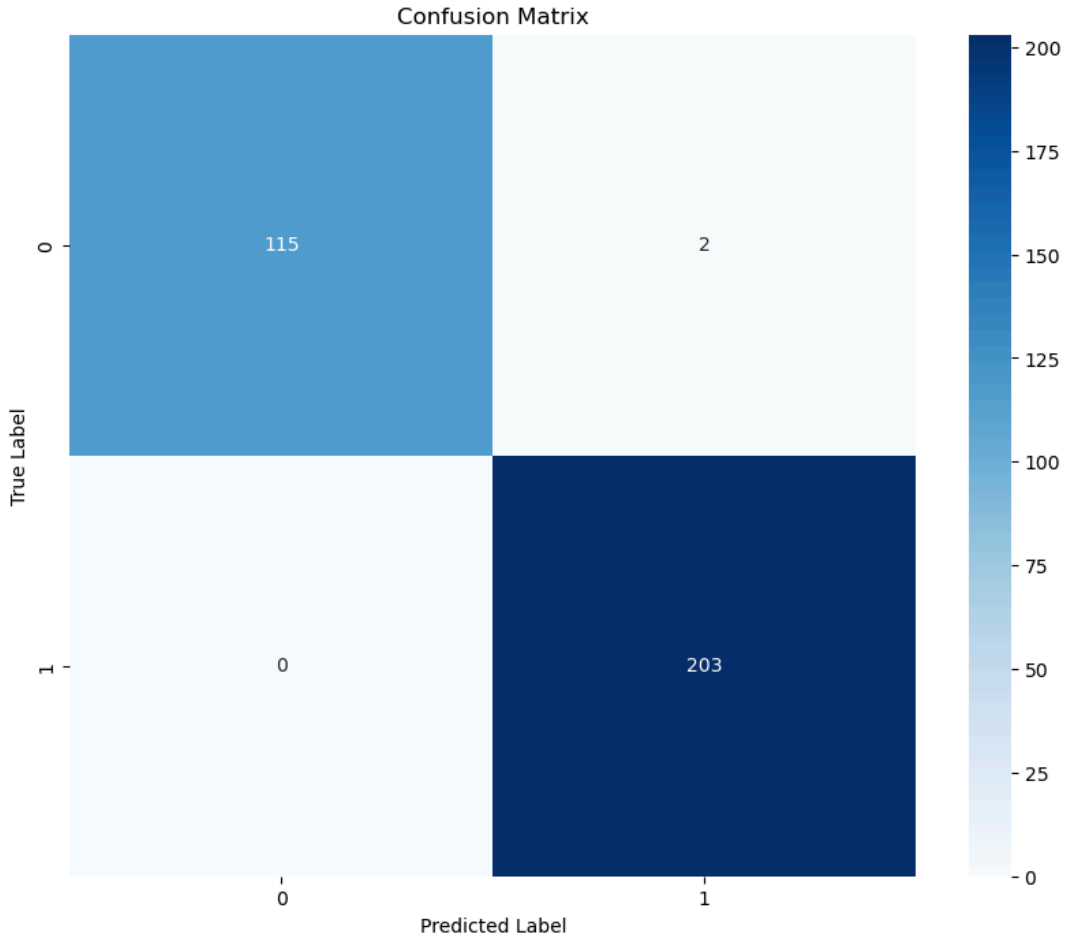
_______________________________________________
ROC AUC Score: 100.00%

_______________________________________________
CLASSIFICATION REPORT:
              precision    recall  f1-score   support

           0       1.00      0.98      0.99       117
           1       0.99      1.00      1.00       203

    accuracy                           0.99       320
   macro avg       1.00      0.99      0.99       320
weighted avg       0.99      0.99      0.99       320


_______________________________________________
Confusion Matrix:
 [[115, 2], [0, 203]]
```

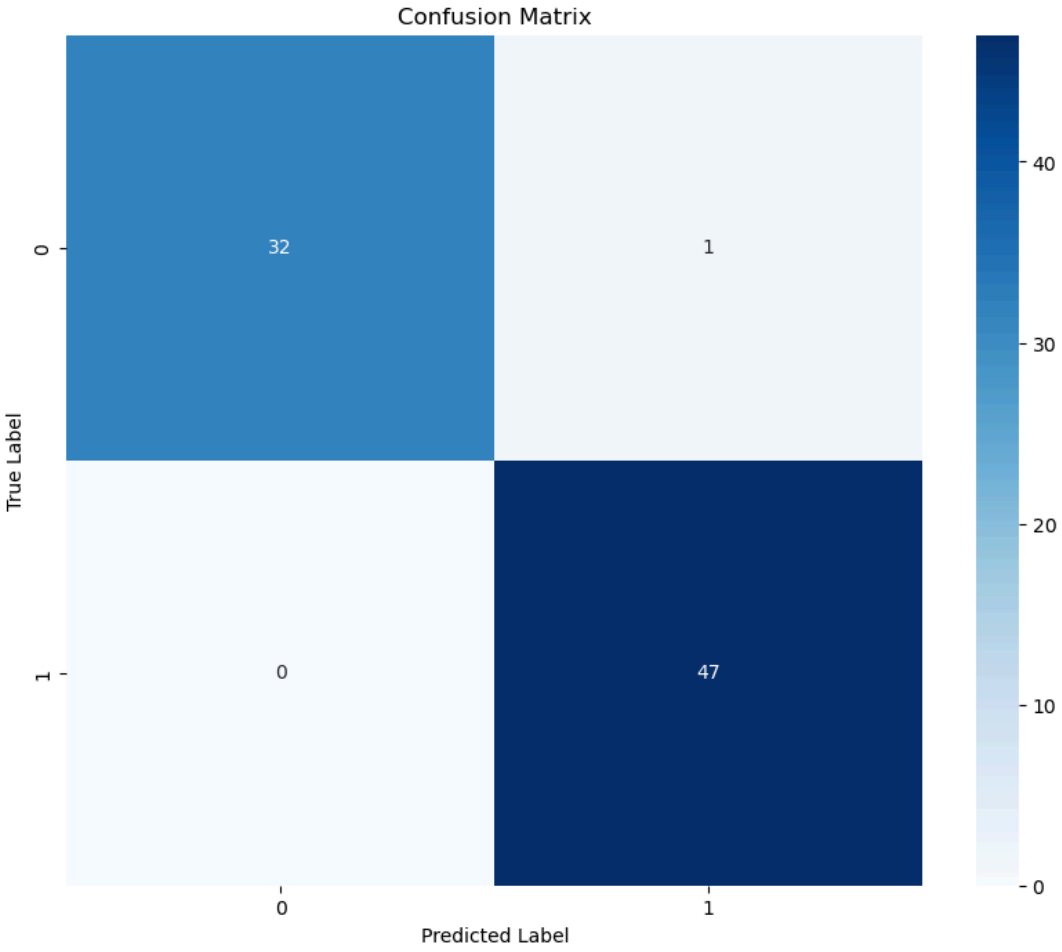


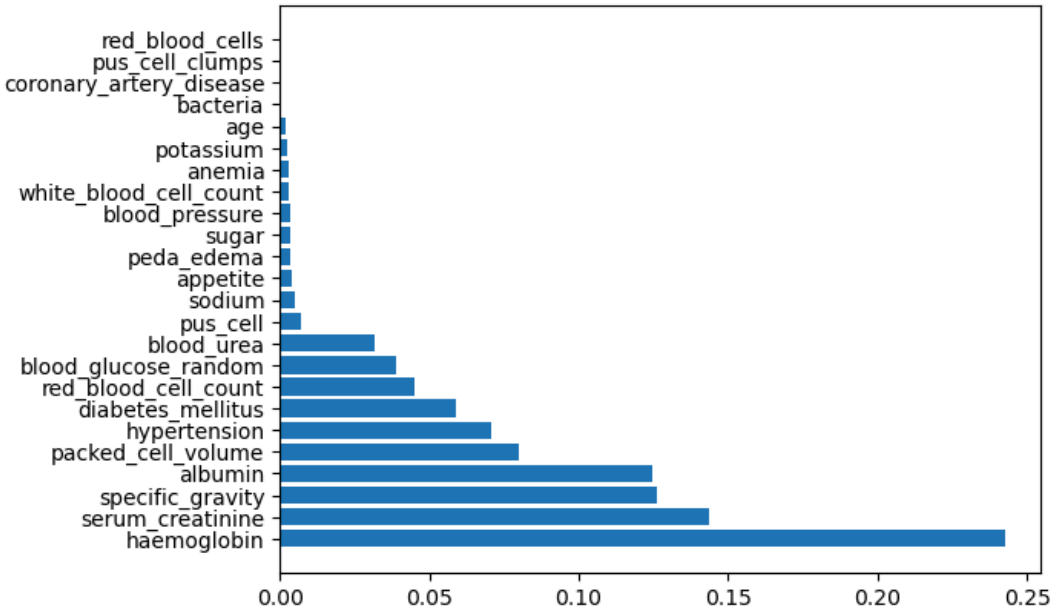

```
Test Result:
================================================
Accuracy Score: 98.75%
_________________________________________________
ROC AUC Score: 99.94%
_________________________________________________
CLASSIFICATION REPORT:
              precision    recall  f1-score   support

           0       1.00      0.97      0.98        33
           1       0.98      1.00      0.99        47

    accuracy                           0.99        80
   macro avg       0.99      0.98      0.99        80
weighted avg       0.99      0.99      0.99        80


_________________________________________________
Confusion Matrix:
 [[32, 1], [0, 47]]
```





Out[66]: ['RandomForestClassifier.pkl']

In [67]:
```
# Table Comparison on models
models = pd.DataFrame({
    'Model' : ['Logistic Regression', 'KNN', 'Decision Tree Classifier'
    'Accuracy Score' : [lr_acc, knn_acc, dtc_acc, rd_clf_acc],
    'ROC/AUC Score' : [lr_ra, knn_ra, dtc_ra, rd_clf_ra]})

models.sort_values(by = 'Accuracy Score', ascending = False)
```

Out[67]:

	Model	Accuracy Score	ROC/AUC Score
3	Random Forest Classifier	98.75	99.935525
0	Logistic Regression	96.25	99.871051
1	KNN	95.00	95.744681
2	Decision Tree Classifier	95.00	97.582205

In [72]:
```
# Create a bar plot
comparison_path=os.path.join(base_dir, 'comparison.png')
plt.figure(figsize=(10, 6))
sns.barplot(data=models.sort_values(by = 'Accuracy Score', ascending =
plt.title('Accuracy Scores of Different Models')
plt.ylabel('Accuracy Score')
plt.tight_layout()
plt.savefig(comparison_path)
plt.show()
```

